EPFL - Computer Science

**COM-402 Information Security Privacy**

Lecture 04 — October 6, 2025

Author: Jonathan Dugout

Professor: Mathias Payer

# Data Security

## Learning Goals

- Understand how access control and encryption are applied in data security
- Learn methods of secure password storage and techniques for password cracking
- Know the basic of Secure Remote Password Protocol (SRP)

## Data Security

Data storage requirements are ubiquitous

- Banking, industry, social networks, government, research, or mobile apps
- Leads to new types of analysis (big data, machine learning)

Companies must consider security of data they store

- What are security requirements of DB systems?
- What are the main attack vectors of DB systems?
- What are main protections of DB systems?

## Typical Setups

- Client ⇒ Internet ⇔ Firewall ⇔ Web server farm ⇔ Database
  - On premises
- Client ⇒ Internet ⇔ Website instances ⇔ Database
  - In the cloud / PaaS (patform-as-a-service)

## Multitier Architecture

- 1) Presentation tier (user interface, web pages, static content). The top-most level of the application. The main function of the interface is to translate tasks and results to something that is intuitive.

- 2) Application (logic) tier. This layer coordinates the application, processes commands, makes logical decisions and evaluations, and performs calculations. It also moves and processes data between the two surrounding layers.
- 3) Data tier (provides data persistence and APIs to access data). Here the information is stored and retrieved from a database or file system. The information is then passed back to the logic tier for processing, and then eventually back to the user.

## Examples of DB Attacks

- Washington state
- Amadeus IT group
- Epic Games (2019-01). All Fortnite player accounts accessible exposing credit card info etc.
- Steam (2018-11) API vulnerability gave access to CD keys for any game. Now patched

# Database: Access Control

## Layers of database

The layers of database accesses:

| Layer | Function | Threat |
|---|---|---|
| Hardware | The disk stores the data of the DB | A thief can take the disk or their backups |
| OS | The user running the DB accesses the files | The sysadmin can access the files |
| Database | DB administrators use privileged DB accounts for maintenance | They can access all data |
| Network | The application tier uses a TCP connection to talk to the DB | Hackers could connect to the DB remotely |
| Application | Uses a DB account to access the data of the application users | An application user can access data of other application users |

Each layer needs to implement proper access control!

## Access Control: Least Privilege

Access control defines which actor can access which resource.

We should apply the principle of **least privilege**

- Each actor can only access the resources strictly needed for its function

This must be applied across all levels of the DB

## Hardware Access Control

If it is a physical machine in a data center

- Physical protection: locks, cameras, alarms

If it is a virtual machine in the cloud

- Cloning the machine is like stealing the hard disk
- Limit the number of people who have the right to clone
- Use strong authentication for your cloud management console (e.g., 2FA)

## OS Access Control

The DB runs as a process in the OS, owned by a certain OS user (e.g., mysql, oracle)

This user is the only one allowed to access the files of the database.

Only user mysql is alloed to access this directory, or any user with root privileges!

## Discretionary DB Access Control

SQL databases use **discretionary** access control (DAC) to grant user access to objects (table and view) through privileges (SELECT, INSERT, UPDATE, DELETE, CREATE, DROP)

By default, the `root` or `system` user has all privileges on all objects.

## Role-Based DB Access Control

SQL databases also support role based access control

Roles are created very much like users

- Privileges can be granted to roles
- Then, roles can be granted to users

E.g., profs can read most columns but only change grades.

You can grant several roles to a same user (employee, lecturer, I&C)

## Network Access Control

Sometimes the application tier runs on the same machine as the DB

- The DB should be configured to only listen to connections from localhost

In other cases, the application tier runs on a different machine

- Accept connections only from machines that should talk to the DB, e.g.,
  - by installing a firewall in front of the DB, and/or
  - by using a local firewall on the DB server, and/or
  - by restricting users to certain IP addresses that can access the DB

## Application Access Control

Applications usually have their own layer of users and privileges

- They use one or few DB accounts to interact with the DB

Example: an e-banking application has 1K customers and three tables:

- A customer table with all customers and their password hashes
- An account table with all accounts and their owners
- A transaction table with all transactions of all accounts

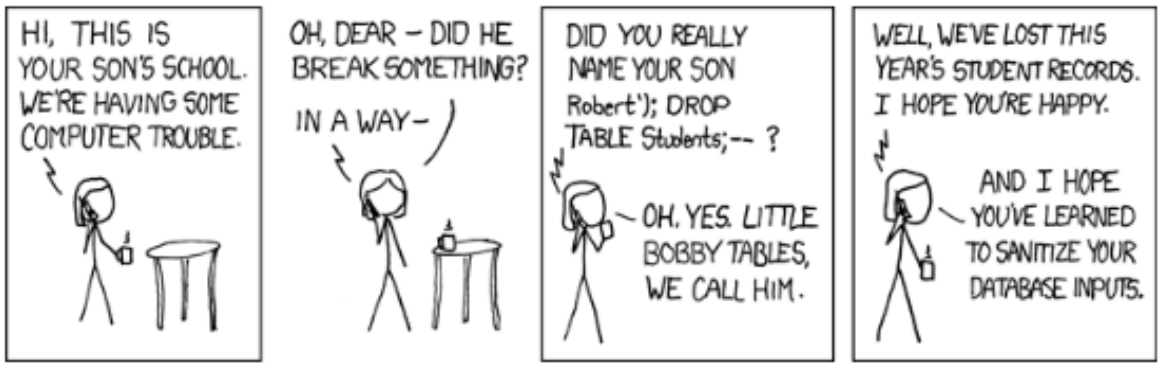
Access control is handled by the application

- It uses the customer table for authentication
- It uses the other two tables to find the accounts and give access to transactions

**Quiz!**

There is a single DB user to access the three tables holding customers, accounts, and transactions
The application makes sure each customer only sees their own data (even if the DB user has access to all data)
What are potential security flaws in this design?

**Discussion**: If the attacker gets the (only) user credentials they could have access to all data. In addition, SQL injection could be really impactful since the user could delete all data.




**Sanitize inputs!!** Use some libraries or other people functions in order to validate user inputs. But for what is SQL, we can use prepared statements. Use structured data with parameters and etc... (like with Spring Boot and SQL statements by giving string parameters)

To limit impact of SQL injections, use different DB users different accesses:

- One DB user with read access on the `customer` table

  - for login purposes
- One DB user with write access on the `customer` table
  - for changing the users' password
- One DB user with read access on the `account` table
  - customers can not change ownership of account
- One DB user with read/write access to the transaction table
  - for the actual application

The database access control is not exactly least privilege, but it reduces the impact of SQL injection (and compartmentalizes code with privileges), e.g., an SQL injection on the login form cannot read data from the `transaction` table

**What's missing for least privilege?**

We're still multiplexing all access on this single DB user. We're still missing the multitier (customers, database users, system administrator, server administrators, ...)

## Application Level Access Control

Using different DB users is an application of the **defense in depth** principle

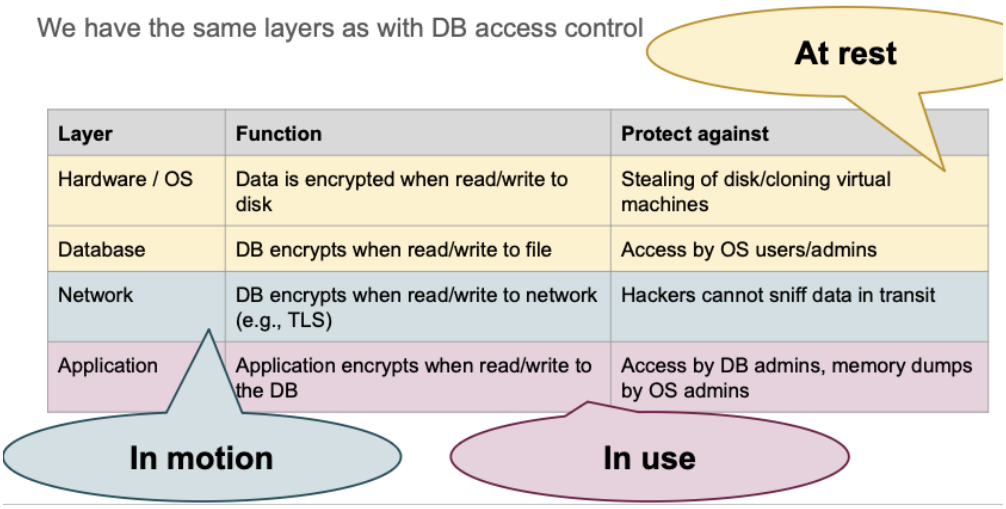
The application implements fine grained access control

The database implements coarse grained control

If the access control at the application level fails, the access control of the database reduces the impact.

**Defense in depth**: multiple layers of security

# Encrypting Data




## Encrypting Data at Rest

Data stored on the server is data at rest. The OS can encrypt the data before it is written to disks

- Protects against theft/copy of the disks
- A user that can log into the machine and access the files of the database will see cleartext data!

Databases can be configured to encrypt data before writing it to files

- File access by OS users does not yield cleartext data anymore
- The keys may be stored in local files or obtained from a key server, e.g., from the Amazon key management service for machines in the Amazon cloud

Most DBs call this type of encryption Transparent Data Encryption (TDE)

**Key use case**: Mobile phones!

## Encrypting Data in Motion

Data is exchanged between the DB and the application (web server or logic tier)

If not encrypted, it could be eavesdropped

Most DBs support TLS encryption for DB connections

- Certificate pinning: the client (DB) can be given a copy of the server certificate or its CA
- It will refuse to connect if the server presents a different certificate (no need for 200 CAs as in web browsers)

For data centers, the bigger cost for encrypting data in motion is power consumption!

## Encrypting Data in Use by the Application

With encryption at rest and in motion, data is still in clear in the memory of the DB!

- AN admin of the DB server can dump the memory and see the data

The solution is to encrypt data in the application before storing it in the DB

The key stays in the application tier

- There is no way to decrypt the data on the DB server

**Quiz: Encrypted Database** What could go wrong if the data is encrypted in the database?

**Discussion**: We lose search capacities and sorting, compare or aggregating data.

This makes the DB pretty useless. But this is still useful for certain information:

- Credit cards numbers
- Pin codes
- Passwords?

# Password Storage

Password leaks are common...

On the darknet you can buy Terabytes of pairs of (username, password).

**Site**: haveibeenpwned.com

## Password Storage

Naive approach: cleartext

- You should **NEVER** store password in cleartext

Better: **hash** the password

- Microsoft stores Windows passwords as hashes (MD4)
- (in MD4) Almost all passwords of length 8 can be recovered in under a minute

## Password Storage: Hashing

Classic way: use **salt** and **iterations** (to stop bruteforcing)

- Hugely slows down password cracking
- Simple passwords can still be cracked on specialized hardware

Modern way: use a **memory hard** function

- Cracking a password requires a decent amount of memory
- Specialized hardware with many computing cores do not have enough memory to speed-up cracking

# The Importance of Salt

SMBv1 (Microsoft) does not salt their passwords. Don't be like Microsoft.

## Salt Usage and Examples

Salt: Random data concatenated as an additional input to the hash function: `pw_hash = h(password, salt)`

Examples

- WPA and WPA2 use the SSID as salt, and 4,096 iterations of HMAC-SHA1
  - Each test becomes expensive
  - Only simple passwords can be cracked
- Kerberos pre-authentication

## The Importance of Salt

Some systems store the passwords as a simple hash: `pw_hash = h(password)`

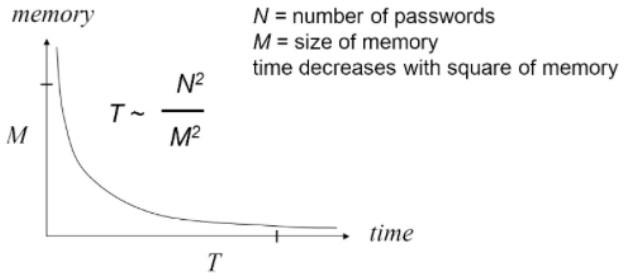
If passwords are hashed without salt, there are two major weakness:

1. Multiple hashes can be cracked at once
   - If you have a list of 1,000 hashes and want to find out if any of the 1,000 accounts has password "correct horse battery staple";
     - calculate `h("correct horse battey staple")`
     - look-up which of the 1,000 hashes matches
   - With a smart data structure, the look-up is almost free, with a single hash operation you can try to crack 1,000 passwords
   - Cracking $n$ passwords requires the same hash operations as cracking one!
2. Hashes can be calculated in advance
   - Windows hashes have no salt
   - If attackers had enough memory, they could calculate all hashes in advance and store them in a great big table
   - The table would be about 4.7PB big! Not feasible
   - Attackers can trade more time of cracking a password for a smaller table

## Time-Memory Trade-Offs (TMTO)

In 1980 Martin Hellman invented a TMTO to inverse cryptographic functions

- When you double the memory, it is four times faster to invert the function




**Rainbow Tables** are an optimization of this TMTO from 2003

## Password Cracking: Time-Memory Trade-Offs

Basic idea: Trade more time of cracking a password for a smaller table

Originally, we can crack one password from one password-hash pair

- Number of passwords: `n`
- Memory required: `n * (password_len + hash_len)`
- Time of cracking password: `time of finding a hash match, i.e., O(c)`

Can we crack two passwords from one password-hash pair?

- Number of passwords: `n`
- Memory required": `1/2 * n * (password_len + hash_len)`

How can we connect the two password-hash pairs?

- Reduction function (**magic function**)
- Map from the hash set to the password set

**Quiz: Password Cracking** What makes a good reduction function?

The reduction function glues the hash of the previous computation to the current one.

Requirements:
- Few collisions
- Output follows the expected alphabet
- Output is reasonably uniformly distributed
- Does NOT require any specific structure

Main idea: reduce memory by half and use computer power with the reduction function to find informations!

More columns, more time of cracking a password, less memory consumption. Typically, chains would contain tens of thousands of hashes and passwords.

| Password | | | | | | | | | | | | | | | | | Hash |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| p0 | h | h0 | r | p7 | h | h7 | r | p4 | h | h4 | r | p17 | h | h17 | r | p47 | h | h47 |
| p1 | h | h1 | r | p64 | h | h64 | r | p23 | h | h23 | r | p6 | h | h6 | r | p32 | h | h32 |
| p2 | h | h2 | r | p11 | h | h11 | r | p42 | h | h42 | r | p35 | h | h35 | r | p9 | h | h9 |
| ... | ... | | ... | | ... | | ... | | ... | | ... | | ... | | ... | | ... | ... |
| pn | h | hn | r | pm | h | hm | r | pt | h | ht | r | pl | h | hl | r | pq | h | hq |

Challenge: Reduction function may collide.

Result in identical chains, which is a waste of table space.

## Password Cracking: Rainbow table

Solution: Using a different reduction function in each column of the table

This allows building much larger tables and makes them much more efficient.

| Password | | | | | | | | | | | | | | | | | Hash |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| p0 | h | h0 | r1 | p7 | h | h7 | r2 | p5 | h | h5 | r3 | p41 | h | h41 | r4 | p33 | h | h33 |
| p1 | h | h1 | r1 | p64 | h | h64 | r2 | p27 | h | h27 | r3 | p76 | h | h76 | r4 | p52 | h | h52 |
| p2 | h | h2 | r1 | p4 | h | h4 | r2 | p22 | h | h22 | r3 | p15 | h | h15 | r4 | p9 | h | h9 |
| ... | ... | | ... | | ... | | ... | | ... | | ... | | ... | | ... | | ... | ... |
| pn | h | hn | r1 | pm | h | hm | r2 | pt | h | ht | r3 | pl | h | hl | r4 | pq | h | hq |

# Storing Hashes with Salt and Iterations

The classical way

## Using Salt

Adding a random value (**salt**) to the hash function prevents the two issues:

- You cannot crack multiple hashes with a single hash calculation
- You cannot calculate the hashes in advance

Because each has a different, random salt

```
salt = random()
pw_hash = h(password, salt)
```

We need to store both the hast and the salt in the database

- When a user logs in, we use the salt to generate a hash and compare it to the stored hash

**Quiz!**
What cryptographic primitive can we use to combine a salt with a hash?
How can we make it harder to crack?

Cryptographic hash functions are used to hash passwords and salt

They are repeated many times to make it harder to crack

## Salt Is Not Enough

Cryptographic hash functions are designed to be fast and simple to implement

A modern graphics card calculates hundreds of billions of hashes per second

A simple way of slowing the attacker is to apply the hash function multiple times

If you require 5,000 iterations for creating the password hash

- Login will take 5,000 times longer (e.g., 0.05s instead of 0.00001s)
- Cracking will be 5,000 times slower (e.g., 2 years instead of 4 hours)

## Salt and Iterations: Standards

There is an official standard for using salt and iterations in hash functions.

The current version is Password Based Key Derivation Function 2 (PBKDF2, RFC 8018)

Used for example in

- Wi-Fi WPA
- MacOS user password hashes

- Linux disk encryption (LUKS)
- Previously for Linux passwords (SHA512, 5000 iterations)

# Memory Hard Hash Function

The modern way of hashing passwords

## Memory Hard Functions

Iterating a cryptographic hash function is not the best way to slow down an attacker

Specialized hardware (graphics cards, FPGAs) computers tera-hashes in parallel

- E.g., it takes less than 50k transistors to implement SHA512

Better password hash functions require a certain amount of memory (e.g., 16MB)

- For a single login operation, 16MB are easily available
- Graphics cards or FPGAs would need gigabytes of internal memory to parallelize thousands of hashes
- E.g., it takes millions of transistors to store 16MB of data

The functions run through many steps and intermediate results are stored in memory. Since each step depends on result from previous steps, if you do not have enough memory you can still calculate the result

- Buy you have to recalculate intermediate values again and again

Typical memory hard hash functions can be parameterized:

- Choose the amount of memory needed
- Choose the number of steps to calculate

**When implementing password storage**

- Always use salt and make the hash function slow
- USe Yescrypt, Scrypt, or Argon2 if available
- If not, use PBKDF2

Most **Linux** distributions switched to yescrypt

## Summary of Password Storage

**NEVER** store passwords in cleartext

Always use **salt** to store passwords. If you do not:

- Multiple hashes can be cracked at once
- Hashes can be calculated in advance

  - Rainbow tables efficiently store this information

Salt is insufficient, must slow down hash function

- Iterations are a good start
  - PBKDF2 is the standard way of doing it
- Memory hard functions are better
  - They are more expensive to parallelize (GPUs and FPGAs have little internal memory)
  - Examples: Yescrypt, Scrypt, or Argon2

# Secure Remote Password Protocol (SRP)

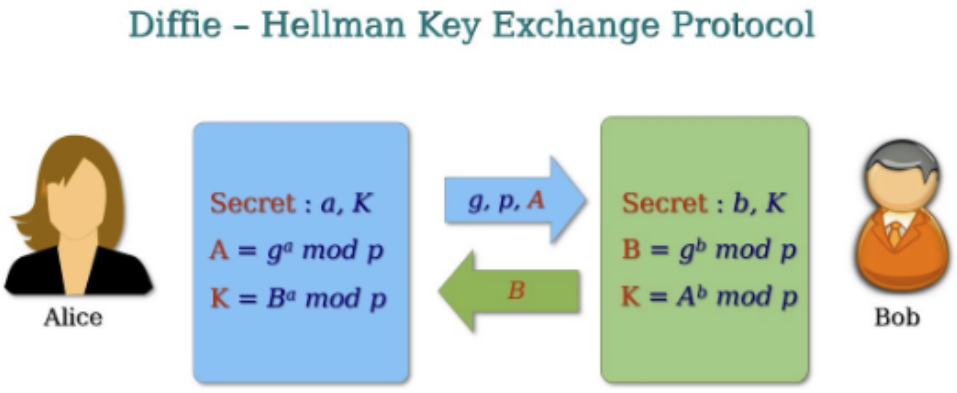
## Introduction

Secure Remote Password protocol (SRP): A Password Authenticated Key Exchange (PAKE)

A PAKE allows to

- Verify the password of a remote party
- Exchange a key (e.g., for encryption)

In TLS, the server can sign its half of the Diffie Hellman key exchange to prove possession of the private key of the certificate, thus proving its identity




PAKE is similar (based on Diffie Hellman), but authentication is based on the password as key where the server stores information ”about” the password

It can’t be cracked, even if the attacker sees all messages.

## SRP Overview

SRP is like DH with additional elements that depend on the password

It uses exponentiations of a generator `g` (e.g., $g^k$) and a modulo $N$

For each user, the server stores three elements:

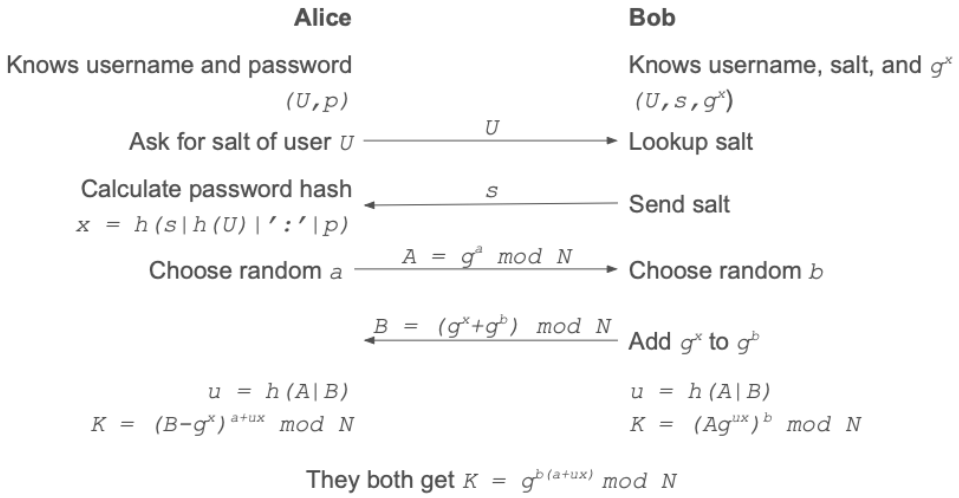
1. The username $U$ 2. A salt $s$ 3. The password verifier $v$ (the exponentiation of a salted hash of the password $p$):"

$$x = h(s|h(U)|':'|p)$$
$$v = g^x \bmod N$$

The server adds $v$ to its part of the Diffie-Hellman exchange

- It contributes to the calculation of the key

The client will need to know the salt to calculate $x$, so it first asks for this




## SRP summary

Alice and Bob have only exchange public values: $g^a$ and $g^b + g^x$

Eavesdropper cannot learn anything from these values

- In particular: they cannot brute-fore the password

The resulting key depends on $a$, $b$ `and` $x$ ($x$ depends on the password)

Bob only knows the public value $g^x$

- If a hacker takes control of Bob, they do not learn the password hash $x$
- They could try to brute force the password from $g^x$

Before continuing, they can send each other an encrypted message or a MAC to prove that they succeeded in calculating the key.